SPENDWISE

Smart Expense Analyzer & Machine Learning Anomaly Detection System

Comprehensive Technical Documentation, System Architecture, Algorithm Training Guide & Viva Voce Q&A; Master Reference

Domain: FinTech & Applied Machine Learning	**Tech Stack:** Python, Streamlit, Scikit-Learn, Plotly
Primary ML Model: Isolation Forest (Unsupervised)	**Target Application:** Personal Finance Intelligence
Core Modules: Ingestion, Analytics, Anomaly ML, Auth	**Storage:** Local Flat-File CSV & JSON (Zero-Cloud Privacy)

Executive Project Summary: *SpendWise is an end-to-end, privacy-preserving financial management system designed to track personal expenditures, discover monthly patterns, and autonomously detect anomalous spending without requiring cloud-based APIs or third-party banking integrations. It implements a Scikit-Learn Isolation Forest ensemble model that isolates mathematical outliers in spending behavior, providing proactive warnings for budget overruns and unusual transactions.*

DOCUMENT TABLE OF CONTENTS

Section 1: Project Overview & Problem Statement	**Section 4:** Isolation Forest ML: Training & Math
Section 2: System Architecture & Flow	**Section 5:** UI Performance & Optimization Fixes
Section 3: Data Pipeline & Schema	**Section 6:** Master Viva Voce Q&A; (30 Questions)

1. Project Overview & Motivation

Modern consumers face significant hurdles maintaining financial discipline. Conventional expense tracking tools suffer from severe operational limitations: (1) manual spreadsheets demand tedious manual formulas and lack intelligence; (2) mobile applications frequently require invasive bank SMS permissions or transmit sensitive financial data to cloud servers; (3) static budgets rely on rigid, hardcoded thresholds (e.g. 'flag if spend > 5,000') that fail to adapt to varied lifestyles.

Core Objectives of SpendWise:

- **Autonomous Anomaly Discovery:** Utilize machine learning to identify statistically improbable transactions rather than static rules.
- **100% Privacy Guarantee:** Local file storage ensures financial data never leaves the user's host environment.
- **Interactive Visual Synthesis:** Dynamic Plotly dashboards enable instant category breakdowns, trends, and daily rhythms.
- **Low Friction Data Ingestion:** Supports flexible CSV imports with automated column aliasing, data type coercion, and manual entry forms.

2. System Architecture & Module Design

SpendWise utilizes a layered modular architecture comprising a Presentation Layer, Business & Analytic Layer, Machine Learning Pipeline Layer, and Data Persistence Layer.

Layer	Components & Libraries	Key Responsibilities
Presentation Layer	Streamlit Multi-Page, BaseWeb UI, Custom CSS	Renders reactive UI, handles session states, form submission, and tab routing.
Analytics Layer	Plotly Graph Objects, Pandas GroupBy	Calculates metrics (daily burn, top categories) and builds interactive charts.
Machine Learning Layer	Scikit-Learn IsolationForest, NumPy	Performs Log1p transformations, fits 100 iTrees, predicts outlier flags.
Data Layer	CSV (`sample_expenses`, `user_expenses`), JSON	Stores raw transactions and user credentials locally on disk.

Application Pages & Workflow:

- **`app.py` (Dashboard Overview):** Aggregates high-level KPIs (Total Spend, Average Daily Spend, Top Category %, Anomalies Flagged), displays Donut chart, spline trend, and recent transaction feeds.
- **`pages/1_Transactions.py`:** Interactive ledger with search filters, category multiselect, date range sliders, and manual transaction entry.
- **`pages/2_Analytics.py`:** In-depth charts for daily expense distributions, category averages, and rule-based Pareto budget insights.
- **`pages/3_ML_Insights.py`:** Displays ordinary vs. unusual histogram overlays, anomaly score cards, and Isolation Forest explanations.
- **`pages/4_CSV_Upload.py`:** Secure drag-and-drop ingestion with header validation, automatic schema renaming, and sample template generator.

- **`pages/5_Settings.py`:** Session termination, sample data reset, permanent data purge, and technical viva documentation download.

3. Data Pipeline & Schema Specification

The system operates on tabular expense records. To ensure compatibility with various bank export formats, the data ingestion engine implements flexible normalization.

Field Name	Target Type	Accepted Aliases	Validation & Constraint
`date`	datetime64[ns]	`transaction_date`, `date`	Parsed via pd.to_datetime; NaT dropped.
`description`	object / string	`merchant`, `description`	Non-empty text string identifying payee.
`category`	category / string	`type`, `category`	Categorical label (Food, Transport, Rent, etc.).
`amount`	float64	`transaction_amount`, `amount`	Numeric value. Inward cash is negative; outward is positive.

Ingestion Cleaning Protocol:

1. Column headers are normalized using `.str.lower().str.strip()`.
2. Header aliases are renamed into standard canonical forms via dictionary mapping.
3. Missing dates or amounts are coerced to NaT/NaN and pruned via `dropna()`.
4. Clean data is serialized to `data/user_expenses.csv`, triggering cache invalidation.

4. Machine Learning Deep Dive: Isolation Forest Algorithm

The core intelligence of SpendWise is powered by the **Isolation Forest** (iForest) algorithm, developed by Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou (2008). Unlike traditional outlier detection algorithms that construct profiles of normal points, Isolation Forest explicitly isolates anomalies.

Why Unsupervised Learning?

Personal expense records lack labeled 'ground truth' indicating fraud or budgetary infractions. Supervised classifiers (Random Forest, Logistic Regression, XGBoost) are unusable because there is no target label column `y`. Isolation Forest requires no prior labels, discovering statistical deviations organically from the unlabeled dataset.

Why Isolation Forest over Alternative Outlier Algorithms?

Algorithm	Computational Complexity	Core Concept / Distance Metric	Verdict for Personal Finance
Isolation Forest	**$O(t \cdot \psi \cdot \log \psi) \approx O(n)$**	Partitions space randomly; anomalies isolate in fewer tree splits.	**Optimal:** Linear complexity, fast, no distance calculations.
One-Class SVM	$O(n^2)$ to $O(n^3)$	Fits a max-margin hyperplane enclosing normal points in kernel space.	Poor scalability; quadratic memory consumption.
DBSCAN	$O(n^2)$	Density clustering; marks unclustered points as noise.	Fails when expense distributions have variable densities.
K-Means Distance	$O(n \cdot k \cdot i)$	Measures Euclidean distance to cluster centroids.	Assumes spherical clusters; requires guessing cluster count k.
LOF (Local Outlier)	$O(n^2)$	Compares local density of a point to its k-nearest neighbors.	Distance calculations slow down drastically as data grows.

Mathematical Theory & Scoring Formulae

1. Tree Construction (iTree): Given a dataset sample X, an isolation tree recursively splits the data. At each node, an attribute q is selected randomly, and a split value p is chosen uniformly between [min(q), max(q)]. Splitting continues until either: (a) tree depth reaches h_max = ceil(log2(ψ)), (b) |X| ≤ 1, or (c) all samples have equal values.
2. Path Length h(x): The path length h(x) is the number of edges traversed from the root node to the terminating leaf node.
3. Average Path Length Baseline c(n): Because iTrees have an equivalent structure to Binary Search Trees (BST), the average path length of an unsuccessful search in a BST over n samples serves as the mathematical normalizer:

Formula 1: Normalizing Constant c(n)
c(n) = 2 * (ln(n - 1) + 0.5772156649) - (2 * (n - 1) / n)
where 0.5772156649 is the Euler-Mascheroni constant (γ).

Formula 2: Anomaly Score s(x, n)
s(x, n) = 2 ^ (- (E(h(x)) / c(n)))
where E(h(x)) is the mean path length across all 100 trees in the ensemble.

Score Interpretation:

- **When s $\to$ 1 (E(h(x)) $\to$ 0):** The observation is isolated in very few splits. It is a definite anomaly.
- **When s < 0.5 (E(h(x)) $\to$ c(n)):** The observation requires typical search depth. It is marked as normal.
- **When all s $\approx$ 0.5:** The entire dataset exhibits uniform density without distinct outliers.

Feature Engineering: Why Log1p Transformation is Mandatory

Financial expenditures follow a heavy-tailed, power-law distribution (Pareto distribution). A typical user might have ninety expenses between Rs 100 - Rs 1,500, and one house rent transaction of Rs 25,000. If raw amounts are used directly, the split range [min, max] is dominated by the single extreme value [100, 25000]. Random splits will overwhelmingly fall in the large empty interval [1500, 25000], causing ordinary mid-range anomalies (e.g. an unexpected Rs 6,500 gadget purchase) to be lumped together with normal expenses.
To resolve this, we apply the natural log transformation: **X = ln(1 + amount)** via `np.log1p()`. This compresses the monetary scale, stabilizes variance, and ensures uniform partition density across order-of-magnitude changes.

Hyperparameters in SpendWise:

- **`contamination = 0.05`:** Pre-specifies that approximately 5% of expenditure records are expected to be unusual. This tunes the decision threshold.
- **`n_estimators = 100`:** Creates 100 distinct random iTrees, ensuring low variance in average path length E(h(x)).
- **`random_state = 42`:** Ensures reproducible trees and deterministic output.
- **`n_jobs = -1`:** Distributes tree construction across all available CPU cores for high execution throughput.
- **Minimum Data Threshold (N $\geq$ 20):** With fewer than 20 points, tree partitioning lacks statistical power. The app automatically disables ML until at least 20 records exist.

5. Performance Profiling & UI Optimization Fixes

During testing, two major system bugs were diagnosed and resolved: an input field invisibility glitch and slow page transition times. Understanding these engineering fixes is frequently examined during technical viva evaluations.

Bug 1: The 'Dark-on-Dark' Input Visibility Defect

Root Cause: In `.streamlit/config.toml`, `secondaryBackgroundColor` was improperly set to `#123C32` (dark forest green). Streamlit's internal BaseWeb design system applies this secondary color to the container wrappers of `st.date_input` and `st.number_input`. Because `base = 'light'`, text was set to `#183B35` (dark green). Consequently, dark green text rendered inside a dark green container, rendering Date and Amount completely invisible.
Engineering Fix: Neutralized `secondaryBackgroundColor` in `config.toml` to `#EFECE6` (light warm cream). In `assets/styles.css`, implemented robust explicit CSS selectors targeting `div[data-baseweb='input']`, `input`, calendar dialogs, and stepper control buttons (`+` and `-`) with solid white backgrounds and high-contrast dark forest typography.

Bug 2: High Latency & Slow Execution Times

Root Cause: Streamlit executes Python scripts sequentially from top-to-bottom on every user click or widget interaction. Without caching: (1) `load_data()` read CSVs from disk and parsed string timestamps on every frame; (2) `detect_anomalies()` re-trained all 100 Isolation Forest trees on every UI rerun; (3) `assets/styles.css` was repeatedly read from disk.
Engineering Fix: Decorated `load_data()`, `detect_anomalies()`, and `get_css()` with `@st.cache_data`. Subsequent page reruns return cached computations in under 1 millisecond. Implemented explicit cache clearing via `clear_data_cache()` and `detect_anomalies.clear()` whenever a transaction is added, uploaded, or deleted.

---

6. Master Viva Voce Q&A; Reference (30 Questions)

This section compiles the exact conceptual, technical, and mathematical questions frequently asked by project examiners.

Q1: What is SpendWise and what real-world problem does it solve?

SpendWise is a privacy-first, machine learning-powered personal expense analyzer. It solves the problem of passive expense tracking by autonomously flagging unusual or anomalous transactions using an Isolation Forest model, while guaranteeing 100% data privacy by keeping all storage and processing strictly local on the user's computer without cloud servers.

Q2: What is the technology stack used in this project?

The project is built entirely in Python. The front-end user interface is developed with Streamlit and styled with Vanilla CSS tokens. Data manipulation is handled by Pandas and NumPy. Visualizations are rendered via Plotly Graph Objects. Machine learning anomaly detection is powered by Scikit-Learn's Isolation Forest.

Q3: Why did you choose Streamlit instead of React or Django?

Streamlit allows rapid prototyping of reactive, data-centric web applications entirely in Python. It seamlessly bridges Python data structures (Pandas DataFrames, NumPy arrays, Plotly figures) with web components, eliminating the need for separate REST API middleware, JSON serialization layers, and JavaScript state sync.

Q4: How does the application guarantee user privacy?

All transactions and credentials are stored strictly in local flat files (CSV and JSON) on the host machine's disk. There are no outbound network calls, no cloud databases (like Firebase or AWS), and no external API integrations. Data remains entirely within the user's control.

Q5: What are the main functional modules of the system?

The system comprises five core modules: (1) Overview Dashboard (KPIs, Donut chart, Trend line), (2) Transaction History (filtering and manual entry), (3) Deep-Dive Analytics (Pareto rules, category burn), (4) Machine Learning Insights (anomaly distributions), (5) CSV Upload Engine, and (6) User Authentication.

Q6: How does the application ingest and validate uploaded CSV files?

When a CSV is uploaded, Pandas reads the buffer. The columns are stripped and lowercased. A dictionary maps common bank aliases (e.g. 'merchant' -> 'description', 'transaction_amount' -> 'amount'). The app verifies that the four required columns exist, converts types with `pd.to_datetime` and `pd.to_numeric`, drops invalid NaNs, and saves the clean file to disk.

Q7: How are income and expense records distinguished in the dataset?

Outward expenditures are stored as positive float values (e.g., +Rs 1,500 for groceries). Inward income or refunds are represented as negative values (e.g., -Rs 50,000 for salary). This standard financial convention allows easy filtering via `df['amount'] > 0` for expense analytics.

Q8: What feature engineering was applied before feeding data to the ML model?

Two essential transformations: First, we filter out all income records (`amount <= 0`) so inward earnings do not skew spending outlier detection. Second, we apply a Log1p transformation: X = ln(1 + amount) using `np.log1p()`. This normalizes the extreme positive skewness characteristic of monetary expenditures.

Q9: What happens if an uploaded CSV file contains invalid or corrupted rows?

The pipeline uses safe type conversion: `pd.to_datetime(..., errors='coerce')` and `pd.to_numeric(..., errors='coerce')`. Any unparseable string is turned into `NaT` or `NaN`. Then `df.dropna(subset=['date', 'amount'])` removes those damaged rows without crashing the application.

Q10: How does the app handle cold-start when a new user has no transaction history?

If no user dataset is detected, SpendWise automatically falls back to a generated synthetic dataset (`sample_expenses.csv`) containing 100 representative transactions with pre-seeded anomalies. If a user uploads an empty dataset with fewer than 20 rows, the ML model gracefully disables itself to prevent invalid predictions.

Q11: Why is anomaly detection in SpendWise formulated as an unsupervised problem?

Personal expense histories do not come with pre-existing labels indicating whether a transaction was 'normal' or 'unusual'. Because ground-truth target labels y do not exist, supervised learning cannot be used. Unsupervised learning identifies anomalies strictly based on intrinsic data properties and distribution geometry.

Q12: How does the Isolation Forest algorithm work fundamentally?

Isolation Forest isolates anomalies instead of modeling normal data. It constructs an ensemble of random binary trees (iTrees). In each tree, features and split values are chosen randomly. Because anomalies are 'few and different', they are isolated near the root of the tree with very short path lengths, whereas normal points require many splits to isolate.

Q13: What is the mathematical definition of path length h(x)?

Path length h(x) is the number of edges an observation x traverses from the root node of an isolation tree to a terminating leaf node. Shorter path lengths correspond to higher anomaly probabilities.

Q14: What is c(n) and why is it needed in the anomaly score formula?

c(n) is the average path length of an unsuccessful search in a Binary Search Tree (BST) built over n observations. It acts as an objective mathematical normalizer so that path lengths across trees and sample sizes can be transformed into a standardized scale between 0 and 1.

Q15: State the formula for the Isolation Forest anomaly score s(x, n).

s(x, n) = 2^(-(E(h(x)) / c(n))), where E(h(x)) is the expected (mean) path length of instance x across all isolation trees in the forest, and c(n) is the average path length normalizer.

Q16: How do you interpret an anomaly score s?

If s is close to 1 (path length much shorter than average), the instance is an anomaly. If s is significantly less than 0.5, the instance is normal. If all instances have s approx 0.5, the dataset does not have distinctly discernible anomalies.

Q17: What does the hyperparameter 'contamination' mean?

Contamination specifies the expected proportion of outliers in the dataset. In SpendWise, it is set to 0.05 (5%). The algorithm uses this parameter to calculate the threshold anomaly score: any transaction with a score in the top 5% is labeled as an anomaly (-1).

Q18: What is n_estimators and what value is used in your project?

n_estimators is the number of isolation trees grown in the ensemble. We set `n_estimators = 100`. Research by Liu et al. demonstrates that 100 trees is the optimal convergence point where path length variance stabilizes with minimal computational overhead.

Q19: Why is an unusual expense NOT necessarily fraudulent?

An anomaly in Isolation Forest simply means mathematical deviation from historical spend patterns. A legitimate rare event—such as paying annual college tuition, booking flight tickets, or buying a new laptop—is mathematically unusual but completely valid. We clearly label flagged transactions as 'Unusual' rather than 'Fraudulent'.

Q20: What is the computational time complexity of training an Isolation Forest?

The training time complexity of Isolation Forest is linear: O(t * psi * log(psi)), where t is the number of trees (100) and psi is the subsampling size. This makes it substantially faster than O(n^2) or O(n^3) distance-based algorithms like DBSCAN or One-Class SVM.

Q21: Why not use K-Means clustering for anomaly detection here?

K-Means requires specifying the number of clusters k in advance, assumes spherical clusters of equal variance, and is severely distorted by outliers because centroids are pulled toward extreme values. Isolation Forest requires no cluster assumption and is explicitly built for outlier isolation.

Q22: Why did you set random_state=42?

Isolation Forest relies on random feature selection and random split choices. Setting a fixed seed (`random_state=42`) ensures determinism and reproducibility, meaning the model produces the exact same anomaly scores on the same dataset across different runs.

Q23: How does Streamlit handle state across multiple page navigations?

Streamlit provides the `st.session_state` dictionary, which persists across page switches and user interactions within the same browser tab session. We store user authentication status (`logged_in: True/False`) and username in `st.session_state` so the login barrier is enforced across all sub-pages.

Q24: What was the cause of the slow page loading and how did you resolve it?

The slowness was caused by re-reading CSV files from disk and re-training the 100-tree Isolation Forest model on every single user click. We resolved this by wrapping `load_data()` and `detect_anomalies()` in `@st.cache_data`. This caches the output in memory, reducing rerun time from over 1 second to under 1 millisecond.

Q25: What is cache invalidation and how is it handled in SpendWise?

Cache invalidation ensures that stale in-memory cached data is refreshed when underlying records change. In SpendWise, whenever a user adds a manual transaction, uploads a new CSV, or deletes data, we explicitly execute `clear_data_cache()` and `detect_anomalies.clear()`, forcing the app to re-fetch and re-fit on the updated dataset.

Q26: What was the Date and Amount input visibility bug and how was it fixed?

Streamlit's `.streamlit/config.toml` had `secondaryBackgroundColor` set to dark green `#123C32`. Streamlit applied this color to the containers of `st.date_input` and `st.number_input`, while text color was also dark green `#183B35`. The dark-on-dark contrast made inputs invisible. We fixed it by changing `secondaryBackgroundColor` to light `#EFECE6` and enforcing CSS white backgrounds with dark text on all `data-baseweb='input'` containers.

Q27: How is Indian Rupee (INR) formatting implemented?

Standard western currency formatting places commas every 3 digits (1,000,000). The Indian numbering system uses the Lakh and Crore system (10,00,000). In `utils/formatting.py`, the `format_inr()` function slices the string: taking the last 3 digits, and then grouping remaining digits in pairs of 2, prepending the 'Rs' symbol.

Q28: How do Plotly charts communicate with the Streamlit frontend?

Plotly creates JSON-serialized declarative figure dictionaries (`go.Figure`). Streamlit renders them using `st.plotly_chart(fig, use_container_width=True)` using the Plotly.js client-side library, allowing tooltips, zooms, and interactive hovering.

Q29: What are the current limitations of this project?

Current limitations include: (1) The ML model evaluates anomaly status primarily on expenditure magnitude rather than multidimensional time-series periodicity; (2) Authentication uses a local JSON dictionary rather than hashed database records (like bcrypt); (3) It currently analyzes single-user data rather than multi-tenant household budgeting.

Q30: What future enhancements would you propose for SpendWise?

Future scope includes: (1) Multi-feature anomaly detection incorporating category frequency, merchant novelty, and day-of-week timing; (2) Time-series expenditure forecasting using Facebook Prophet or ARIMA; (3) OCR receipt scanning using Tesseract to automatically ingest paper bills; (4) Automated budget alert emails or Telegram push notifications.

End of Viva Defense Guide. *Prepared for academic project submission and oral examination.*